

Codeforces 865D — Buy Low Sell High

Problem Link: codeforces.com/problemset/problem/865/D

Difficulty: 2000 (Constructive Algorithms / Greedy)

Time Limit: 2.0 seconds

Memory Limit: 256 MB

1. Problem Statement

You are perfectly predicting the price of a certain stock for the next N days. On the i -th day, the price of the stock will be P_i .

Each day, you can perform one of three actions:

- **Buy** one share of the stock at the current price P_i .
- **Sell** one share of the stock that you currently hold at the current price P_i .
- **Do nothing** (neither buy nor sell).

Initially, you hold 0 shares of the stock, and you cannot have a negative balance of shares at any point (i.e., you cannot short sell). What is the maximum profit you can achieve by the end of the N days?

Constraints

- $1 \leq N \leq 3 \cdot 10^5$
- $1 \leq P_i \leq 1 \cdot 10^6$

2. The Core Dilemma & Dynamic Hurdle

A naive greedy strategy might suggest that whenever we see a price higher than a previous buying price, we should take the profit. However, a local optimization can lock us out of a globally optimal decision.

Consider the price sequence: `[2, 5, 9]`.

- If we buy at `2` and immediately sell at `5`, we secure a profit of $5 - 2 = 3$.
- However, because we sold our only share on Day 2, we cannot exploit the massive spike to `9` on Day 3. The optimal path is to buy at `2` and hold until Day 3 to sell at `9`, achieving a profit of $9 - 2 = 7$.

The core challenge is that when we are on Day 2, we do not know whether `5` is the highest achievable peak or just a minor bump on the way to a higher peak. Standard backtracking or dynamic programming would require looking into the future, scaling poorly for $N = 3 \cdot 10^5$.

3. The "Options & Buy-Back" Framework

To formulate an efficient linear-logarithmic approach, we redefine our actions using **Financial Options Theory** accompanied by a ****regret mechanism****.

Concept A: Accumulating Options

Instead of physically purchasing a stock on Day i and locking up capital, imagine the game awards you a free, non-binding **option (coupon)** every single day. This option grants you the permanent right to buy exactly one share of stock at that day's price P_i at any point in the future. Because it is an option and not an obligation, you incur zero risk by simply storing it in your collection.

Concept B: Risk-Free Exploitation

On each current day i with price P_i , we check our collection of options. If the absolute cheapest option in our repository has a strike price P_{old} that is strictly less than P_i , we can guaranteed a profit. We immediately exercise that option and sell it at today's market value.

$$\text{Immediate Profit} = P_i - P_{old}$$

Concept C: The Buy-Back Clause (Undoing Regret)

What if selling today at P_i turns out to be a suboptimal decision because a higher price P_{future} appears later?

To circumvent this, every time we choose to exercise an old option and sell at P_i , we immediately hand ourselves a brand-new **buy-back option** with a strike price equal to today's price P_i .

If a future day j presents a price $P_{future} > P_i$, we can choose to exercise this buy-back option. Let's look at the mathematical synthesis of this chained transaction:

$$\text{Total Profit} = (P_i - P_{old}) + (P_{future} - P_i)$$

Simplifying the expression:

$$\text{Total Profit} = P_{future} - P_{old}$$

Crucial Insight: The intermediate term P_i completely cancels out. By exercising the buy-back option in the future, the system naturally retroactively transforms the transaction sequence. It functions exactly as if we never sold the stock on Day i , but instead held the original share bought at P_{old} all the way until the future date to sell at P_{future} .

4. Algorithmic Structure via Min-Heap

Because our greedy strategy dictates that we only ever care about finding and extracting the absolute **cheapest** option available in our repository, a **Min-Heap (Priority Queue)** is the optimal data structure.

Daily Invariant Execution Steps:

For each day's price P_i from Day 1 to Day N :

1. Check if the min-heap is non-empty and its root element P_{top} (the cheapest available option) satisfies $P_{top} < P_i$.
2. If $P_{top} < P_i$:
 - We accumulate profit: $Profit \leftarrow Profit + (P_i - P_{top})$.
 - Pop P_{top} from the heap (it has been consumed).
 - Push P_i into the min-heap. This represents the *buy-back option* allowing future states to undo this sell.
3. **Always (Regardless of whether a transaction happened):**
 - Push P_i into the min-heap. This represents the standalone, base option generated natively by the current day.

Note on Element Duplication: If a transaction occurs, P_i is pushed into the heap **twice**. Once acting as a placeholder/buy-back for the option we just modified, and once as a new day option.

5. Detailed Step-by-Step Simulation

Let's simulate the problem's trickiest state sequence: Prices = [2, 5, 9]

Day (i)	Price (P_i)	Heap State Before	Action Taken & Calculations	Heap State After	Running Profit
1	2	[] (Empty)	Heap empty. Push base option 2.	[2]	0
2	5	[2]	Top (2) < 5. Profit += (5 - 2) = 3. Pop 2. Push buy-back 5, push base 5.	[5, 5]	3
3	9	[5, 5]	Top (5) < 9. Profit += (9 - 5) = 4. Pop one 5. Push buy-back 9, push base 9.	[5, 9, 9]	7

Mathematical Verification: The total profit evaluated is $3 + 4 = 7$. Looking at the remaining values in the heap (`[5, 9, 9]`), we see that one instance of `5` was never modified. The net transaction equates exactly to buying at `2` and selling at `9`. The buy-back mechanism elegantly resolved the temporal decision tree without complex state tracking.

6. Complexity Analysis

- **Time Complexity:** $O(N \log N)$

We process each of the N days via a single bounded loop. Inside the loop, we perform standard binary heap pushes and pops. Since the heap size is capped tightly at $2N$ elements, each priority queue operation costs at most $O(\log N)$ time. The total processing time easily scales within the 2.0-second time limit for $N = 3 \cdot 10^5$.

- **Space Complexity:** $O(N)$

In the absolute worst-case scenario (e.g., a strictly decreasing asset price line), no elements are ever popped from the heap, leading to a maximal structural footprint bounded linearly by N .

7. Final Verified Solution (C++)

Below is the complete, high-performance implementation utilizing C++ Standard Template Library's `std::priority_queue`.

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

int main() {
    // Optimize standard I/O operations for competitive programming
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    if (!(cin >> n)) return 0;

    // Min-heap tracking available stock buying options
    priority_queue<long long, vector<long long>, greater<long long>> min_heap;
    long long total_profit = 0;

    for (int i = 0; i < n; ++i) {
        long long current_price;
        cin >> current_price;

        // If a valid profit can be extracted using our cheapest option
```

```

        if (!min_heap.empty() && min_heap.top() < current_price) {
            total_profit += (current_price - min_heap.top());
            min_heap.pop(); // Consume the option

            // Push current price as a buy-back option
            min_heap.push(current_price);
        }

        // Always push the current price as a primary day choice
        min_heap.push(current_price);
    }

    cout << total_profit << "
";

    return 0;
}

```